Document Versioning System

1. Project Overview

This project implements a web-based Document Versioning System that allows users to create, edit, and manage documents with full version history.
Each save operation creates a new immutable version. Users can browse all versions, preview any old version without modifying the current one, roll back to any previous version, and delete unwanted versions (at least one version is always kept).The system follows a single-page application (SPA) architecture.

---

2. Technology Stack

Layer	Technology
Backend	Python 3.10+, FastAPI, PyMongo
Database	MongoDB (or mongomock for testing)
Frontend	HTML5, CSS3, Vanilla JavaScript, Editor.js
Server	Uvicorn

---

3. API Structure

Method	Endpoint	Description
GET	`/api/documents`	List all documents (title, current version info)
POST	`/api/documents`	Create a new document (body: `{"title": "..."}`)
DELETE	`/api/documents/{document_id}`	Delete a document and all its versions
GET	`/api/documents/{document_id}`	Get document metadata + current version content
GET	`/api/documents/{document_id}/versions`	List all versions of a document (id, revision, date)
GET	`/api/documents/{document_id}/versions/{version_id}`	Retrieve content of a specific version (preview)
POST	`/api/documents/{document_id}/versions`	Save current editor content as a new version
DELETE	`/api/documents/{document_id}/versions/{version_id}`	Delete a version
POST	`/api/documents/{document_id}/rollback/{version_id}`	Rollback the document to a previous version

---

4. Code Structure

Because the frontend is an SPA, the backend follows the Model-Controller-Service pattern:

```
project/
├── main.py           # Controller (route handlers)
├── storage.py        # Service layer (all DB operations)
├── db.py             # Database connection & index setup
├── schemas.py        # Models (Pydantic schemas)
├── static/
│   └── app.css       # Styling
└── templates/
    └── editor.html   # Frontend SPA (HTML/JS)
```

Model (schemas.py): Pydantic classes (`DocumentOut`, `VersionOut`, `DocumentCreate`, etc.) define the shape of data exchanged with clients.

Controller (main.py): FastAPI route functions handle HTTP requests, call the service layer, and return responses.

Service (storage.py): Contains all business logic and MongoDB operations (creating documents, saving versions, rollback, deletion, etc.).

---

5. Database Design (MongoDB)

Collection: `documents`

Field	Type	Description
`_id`	ObjectId	Unique document identifier
`title`	string	Document title (1-200 characters)
`current_version_id`	ObjectId	`_id` of the currently active version (may be `null` initially)
`current_version_revision`	int	Revision number of the current version (0 if none, redundant for quick display)
`created_at`	string	ISO timestamp of creation

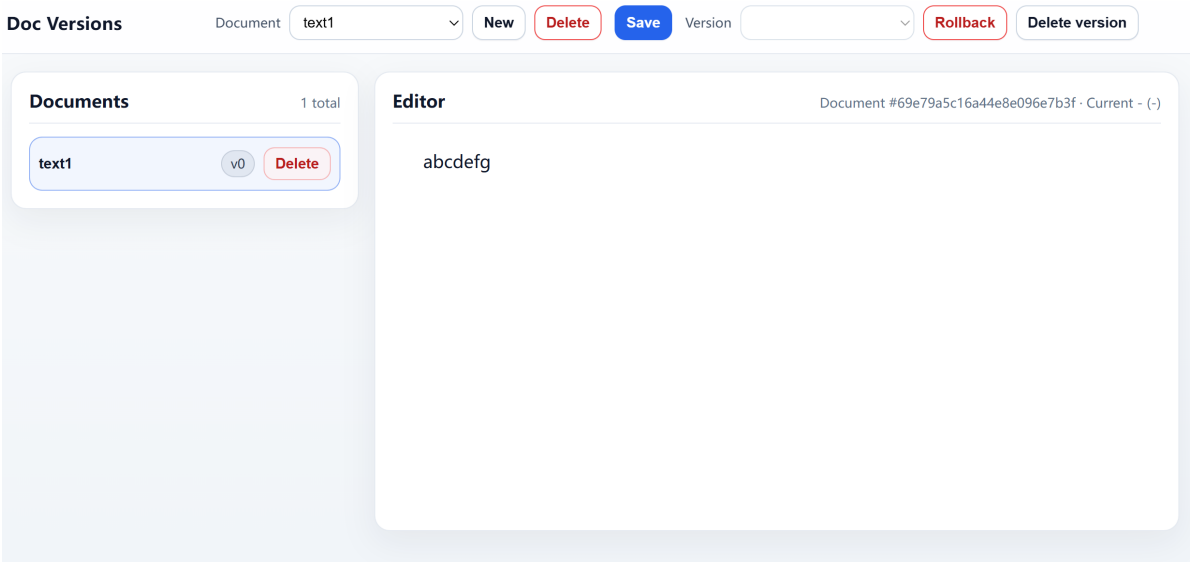
Collection: `document_versions`

Field	Type	Description
`_id`	ObjectId	Unique version identifier
`document_id`	ObjectId	Reference to the parent document (required)
`revision`	int	Sequential version number (1, 2, 3, ...)
`content`	object	Editor.js JSON object (blocks, time, version)
`created_at`	string	ISO timestamp of version creation

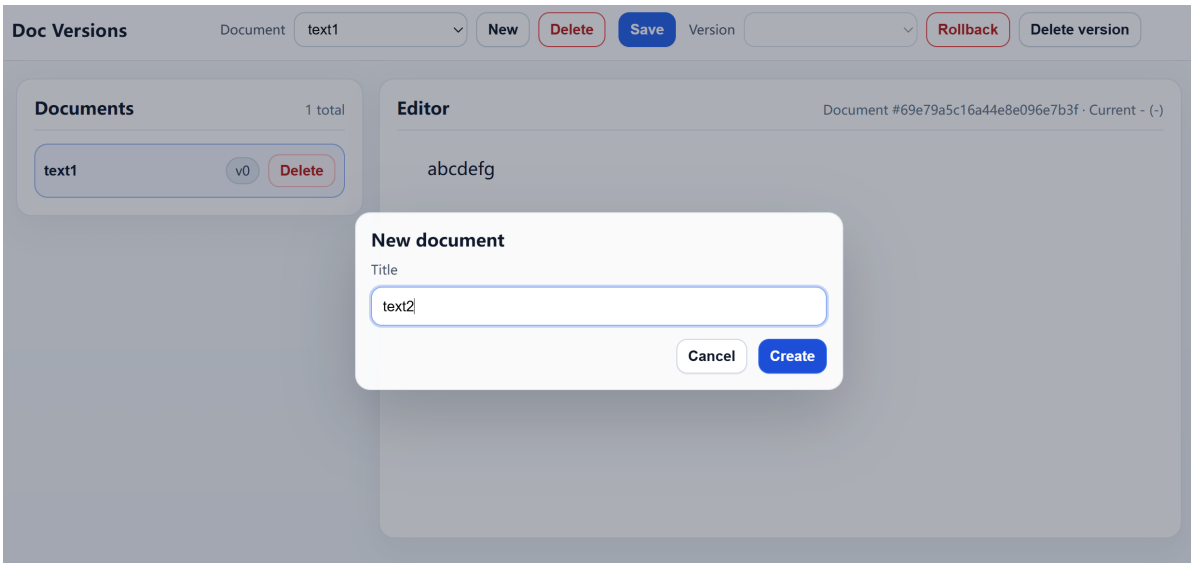
---

6. User Interface

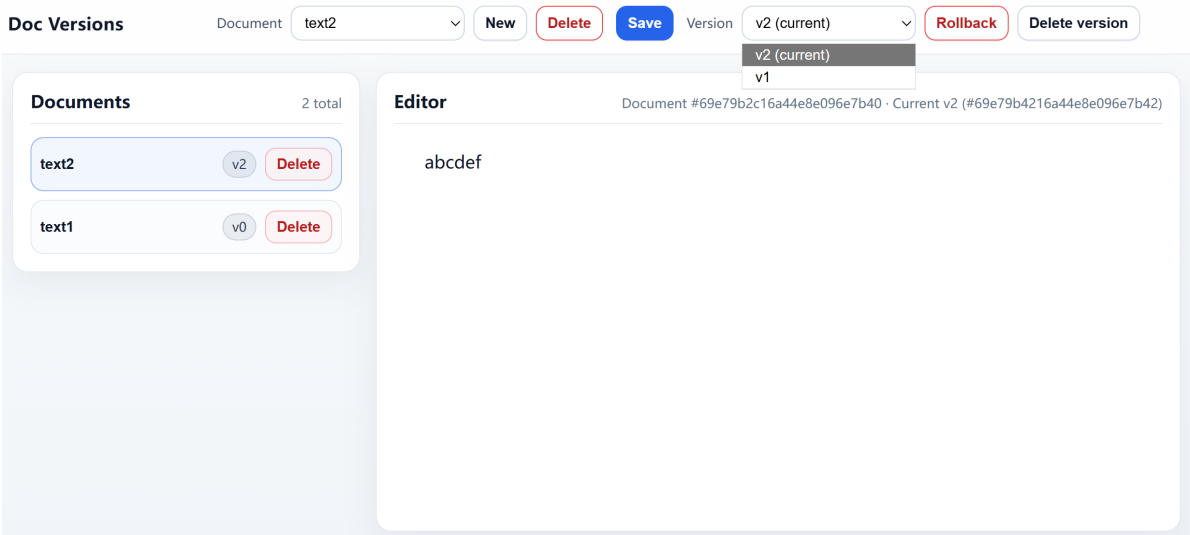
Left sidebar shows all documents with their current version numbers. Top toolbar allows switching documents, creating new ones, and deleting the current document. The main area contains the Editor.js block editor.



Click "New" to create a new document.



The “Save” button creates a new version. Select any version from the dropdown to preview its content. The “Rollback” button makes that version the new current version.



Users can delete old versions, but the last remaining version cannot be removed.

7. How to Run the Application

Install dependencies:

```
pip install -r requirements.txt
```

Set environment variables to tell the program to use the mock database.

```
set MONGODB_URI=mongomock://localhost
```

Start the application.

```
cd app
uvicorn main:app --reload
```

Open browser at `http://localhost:8000`

Student ID: 508658
Name: Lei Yaqi